

ADP(데이터분석 전문가) 실기시험 준비자료

2부 — 5단계 군집분석 · 6단계 시계열분석 · 7단계 텍스트마이닝 · 8단계 종합모의고사

[사용자] 네 진행해줘

ADP 실기시험 대비 자료 (5) — 비지도학습: 군집분석(Clustering)

군집분석은 목표변수(정답)가 없는 **비지도학습**이므로, 지도학습과는 답안 작성 방식이 다릅니다. 특히 "몇 개의 군집으로 나눌 것인가"에 대한 근거 제시와 군집별 특성(프로파일링) 해석이 핵심 배점 포인트입니다.

핵심 체크리스트

항목	확인할 것
스케일링	K-means 등 거리 기반 알고리즘은 변수 간 단위 차이가 결과를 왜곡하므로 표준화 필수
최적 군집 수 결정	엘보우 기법(Elbow Method) + 실루엣 계수(Silhouette Score) 두 가지를 함께 제시하면 근거가 탄탄해짐
알고리즘 선정	군집이 구형(spherical)이라 가정 가능하면 K-means, 군집 형태가 불규칙하거나 이상치가 많으면 DBSCAN, 계층 구조를 보고 싶으면 계층적 군집
군집 결과 시각화	변수가 3개 이상이면 PCA로 2차원 축소 후 산점도로 시각화
군집별 프로파일링	각 군집의 변수별 평균/특성을 비교하여 "이 군집은 어떤 고객군인가"를 이름 붙여 해석 (매우 중요한 서술 포인트)
실무적 활용방안	군집별 특성에 따른 차별화된 전략 제안까지 서술하면 고득점

답안 작성 표준 틀

- ① 스케일링 근거 서술
- ② 최적 군집 수 결정 (엘보우 + 실루엣 계수, 근거와 함께 K 확정)
- ③ 군집 모델 학습 및 결과 부여
- ④ 군집별 특성 프로파일링 (표로 정리) 및 시각화
- ⑤ 각 군집에 이름을 붙이고 실무적 해석/활용방안 제시

실기유형 연습문제 (8) — K-means 군집분석 및 고객 세분화

※ 아래 문제는 실제 시행된 시험의 출제 패턴(고객 세분화, RFM 분석 등 마케팅 데이터를 활용한 군집분석 구조)을 반영하여 재구성한 기출유형 연습문제입니다.

[문제]

다음은 어느 유통업체의 고객 구매 데이터(rfm.csv)이다.

- recency: 최근 구매일로부터 경과일수 (작을수록 최근에 구매)
- frequency: 최근 1 년간 구매 횟수
- monetary: 최근 1 년간 총 구매금액(만원)

(5-1) 데이터를 표준화하고, 엘보우 기법과 실루엣 계수를 활용하여 최적의 군집 수를 결정하시오.

(5-2) 결정한 군집 수로 K-means 군집분석을 수행하고 결과를 시각화하시오. (5-3) 각 군집의 특성을 분석하여 고객군의 이름을 부여하고, 군집별 마케팅 전략을 제안하시오.

모범 답안

(5-1) 표준화 및 최적 군집 수 결정

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

df = pd.read_csv('rfm.csv')

# 1) 표준화 (recency, frequency, monetary는 단위와 스케일이 크게 다르므로 필수)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df[['recency', 'frequency', 'monetary']])

# 2) 엘보우 기법
sse = []
K_range = range(1, 11)
for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    sse.append(km.inertia_)

plt.figure(figsize=(6,4))
plt.plot(K_range, sse, marker='o')
plt.xlabel('군집 수 (K)')
plt.ylabel('SSE (군집 내 분산)')
plt.title('엘보우 기법을 통한 최적 K 탐색')
plt.show()

# 3) 실루엣 계수 (K=2~6 구간 비교)
sil_scores = []
for k in range(2, 7):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)
    sil_scores.append(score)
```

```
print(f"K={k}, 실루엣 계수: {score:.4f}")
```

[서술] recency, frequency, monetary는 측정 단위와 값의 범위가 서로 크게 달라, 거리 기반 알고리즘인 K-means 적용 전 **표준화(StandardScaler)**를 수행하였다. 엘보우 기법 확인 결과 K=4 부근에서 SSE 감소폭이 완만해지는 elbow point가 관찰되었으며, 실루엣 계수 역시 K=4에서 상대적으로 높은 값을 보여 **두 지표가 일치하는 결과를 근거로 최적 군집 수를 K=4로 결정**하였다. (실루엣 계수는 -1~1 범위를 가지며 1에 가까울수록 군집 내 응집도와 군집 간 분리도가 모두 우수함을 의미한다.)

(5-2) K-means 군집분석 수행 및 시각화

```
from sklearn.decomposition import PCA

# 최종 모델 (K=4)
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
df['cluster'] = kmeans.fit_predict(X_scaled)

# 시각화를 위한 PCA 2차원 축소 (변수가 3개이므로 2차원으로 압축하여 시각화)
pca = PCA(n_components=2)
pca_result = pca.fit_transform(X_scaled)
df['pca1'] = pca_result[:, 0]
df['pca2'] = pca_result[:, 1]

print(f"PCA 설명 분산 비율: {pca.explained_variance_ratio_}")

plt.figure(figsize=(7,5))
scatter = plt.scatter(df['pca1'], df['pca2'], c=df['cluster'], cmap='viridis')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.title('K-means 군집 결과 (PCA 2차원 시각화)')
plt.colorbar(scatter, label='군집')
plt.show()
```

[서술] 3개의 변수(recency, frequency, monetary)를 시각적으로 확인하기 위해 **PCA로 2차원 축소** 후 산점도로 표현하였다. PCA 설명 분산 비율의 합이 충분히 높게 나타나(예: 85% 이상), 2차원으로 축소하더라도 원 데이터의 정보 손실이 크지 않음을 확인하였다. 시각화 결과 4개의 군집이 비교적 뚜렷하게 분리되어 나타나, K=4로 결정한 군집화가 타당함을 시각적으로도 뒷받침하였다.

(5-3) 군집별 프로파일링 및 마케팅 전략 제안

```
# 군집별 변수 평균 비교 (프로파일링)
profile = df.groupby('cluster')[['recency', 'frequency', 'monetary']].mean().round(1)
profile['count'] = df['cluster'].value_counts().sort_index()
print(profile)
```

[결과 예시 및 서술]

cluster	recency	frequency	monetary	count
0	15.2	18.5	450.3	120

cluster	recency	frequency	monetary	count
1	8.1	22.0	680.5	95
2	95.4	3.2	80.1	150
3	45.0	8.5	210.2	135

[서술] 군집별 프로파일링 결과를 바탕으로 각 군집에 다음과 같이 이름을 부여하고 전략을 제안하였다.

- **군집 1 (VIP 우수고객):** recency 가 가장 낮고(최근 구매) frequency·monetary 가 가장 높은 그룹. → 로열티 프로그램, 프리미엄 혜택 제공을 통한 유지 전략이 적합하다.
- **군집 0 (일반 우량고객):** 군집 1 보다는 다소 낮지만 준수한 구매 활동을 보이는 그룹. → 교차판매(Cross-selling), 업셀링(Up-selling) 프로모션을 통해 VIP 로 전환을 유도할 수 있다.
- **군집 3 (관심 필요 고객):** 구매 빈도와 금액이 낮아지고 있는 그룹. → 재구매를 유도하는 리마인드 메일, 할인 쿠폰 발송 등 이탈 방지 전략이 필요하다.
- **군집 2 (이탈 위험/휴면 고객):** recency 가 매우 크고(오랫동안 구매하지 않음) frequency·monetary 가 가장 낮은 그룹. → 윈백(win-back) 캠페인을 통한 재활성화를 시도하되, 반응이 없으면 마케팅 비용 효율화 관점에서 우선순위를 낮추는 것을 고려할 수 있다.

이처럼 RFM 기반 군집분석 결과를 활용하면 **전체 고객에게 동일한 마케팅을 적용하는 대신, 고객 군별 맞춤형 전략을 수립**할 수 있다는 실무적 시사점을 제시할 수 있다.

실기유형 연습문제 (9) — 계층적 군집분석과 K-means 비교

[문제]

위 데이터에 대해 계층적 군집분석(Hierarchical Clustering)을 수행하고 덴드로그램을 시각화하시오. 또한 K-means와 계층적 군집분석의 결과 차이 및 각 방법의 장단점을 서술하시오.

모범 답안

```
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import AgglomerativeClustering

# 와드연결법(Ward's method) - 군집 내 분산 증가를 최소화하는 방식으로 가장 널리 사용됨
Z = linkage(X_scaled, method='ward')

plt.figure(figsize=(10,5))
dendrogram(Z, truncate_mode='lastp', p=20)
plt.title('계층적 군집분석 덴드로그램 (Ward Linkage)')
```

```
plt.xlabel('데이터 포인트(또는 군집)')
plt.ylabel('거리(유클리드 기준)')
plt.show()

# 덴드로그램에서 결정한 군집 수(K=4)로 계층적 군집 수행
hc = AgglomerativeClustering(n_clusters=4, linkage='ward')
df['cluster_hc'] = hc.fit_predict(X_scaled)

# K-means와 계층적 군집 결과 일치도 비교 (교차표)
print(pd.crosstab(df['cluster'], df['cluster_hc']))
```

[서술] 와드연결법(Ward Linkage)은 군집을 병합할 때 **군집 내 분산의 증가폭을 최소화**하는 기준으로 병합하는 방식으로, 계층적 군집분석에서 가장 널리 사용된다. 덴드로그램을 통해 4개 군집으로 절단했을 때의 구조를 확인하였고, K-means 결과와의 교차표를 통해 두 방법의 군집 배정이 상당 부분 일치함을 확인하였다.

[방법 비교 서술]

- **K-means:** 사전에 군집 수(K)를 지정해야 하며, 군집이 구형(spherical)이고 크기가 비슷하다는 가정 하에 잘 작동한다. 대용량 데이터에도 계산 효율이 좋다는 장점이 있으나, 초기 중심값에 따라 결과가 달라질 수 있고 이상치에 민감하다는 단점이 있다.
- **계층적 군집분석:** 사전에 군집 수를 지정할 필요 없이 덴드로그램을 통해 다양한 군집 수의 결과를 한번에 탐색할 수 있다는 장점이 있으나, 데이터 크기가 커질수록 계산 복잡도가 급격히 증가하여(모든 데이터 쌍 간 거리 계산 필요) 대용량 데이터에는 부적합하다.

[결론] 본 데이터는 규모가 크지 않아 두 방법 모두 적용 가능하였으며, 두 결과가 유사하게 나타난 것은 **군집 구조가 비교적 명확하다는 것을 시사**한다. 실무에서는 탐색 단계에서는 계층적 군집 분석으로 전체적인 구조를 파악하고, 최종 대량 데이터 적용 시에는 K-means를 사용하는 방식으로 두 기법을 상호 보완적으로 활용하는 것이 합리적이다.

이 파트에서 감점을 피하는 핵심 포인트

- **✗** 스케일링 없이 바로 K-means 적용 → 변수 단위가 다르면 거리 계산이 왜곡되어 반드시 표준화 선행
- **✗** 군집 수(K)를 근거 없이 임의로 설정 → 엘보우 기법과 실루엣 계수를 함께 제시하여 근거를 명확히 할 것
- **✗** 군집을 나누기만 하고 프로파일링(해석) 없이 끝냄 → 각 군집에 이름을 붙이고 특성을 설명하는 것이 배점의 핵심
- **✗** 군집 결과를 마케팅/실무 전략과 연결하지 않음 → 분석 결과가 "그래서 어떻게 활용할 수 있는지"까지 서술해야 완성된 답안
- **✓** K-means 와 다른 군집기법(계층적/DBSCAN)을 비교하며 각각의 장단점을 논하면 통계적 이해도 측면에서 고득점

ADP 실기시험 대비 자료 (6) — 시계열 분석

시계열 분석은 매 회차 고정적으로 출제되지는 않지만, 출제될 경우 정상성 검정 → 차분 → 모형 식별(ACF/PACF) → 모형 적합 → 잔차 진단 → 예측이라는 정형화된 흐름을 정확히 따라야 고득점할 수 있는 파트입니다.

핵심 체크리스트

항목	확인할 것
시각화 우선	원 시계열을 그려 추세/계절성/이상치를 먼저 육안으로 확인
정상성 검정	ADF(Augmented Dickey-Fuller) 검정 — p-value < 0.05이면 정상 시계열로 판단
차분	비정상이면 1차 차분 후 재검정, 계절성이 있으면 계절차분 추가
모형 식별	ACF/PACF 플롯을 통해 AR(p), MA(q) 차수 추정 (또는 auto_arima로 자동 탐색)
모형 적합	statsmodels의 ARIMA 또는 pmdarima의 auto_arima
잔차 진단	잔차가 백색잡음(White Noise)인지 Ljung-Box 검정으로 확인 — 잔차에 자기상관이 남아있으면 모형이 불완전
예측 및 평가	미래 예측 후 실제값과 비교(가능한 경우), MAE/RMSE/MAPE로 평가

답안 작성 표준 틀

- ① 시계열 시각화 및 특성(추세/계절성) 관찰
- ② 정상성 검정(ADF) → 필요시 차분 수행 및 재검정
- ③ ACF/PACF를 통한 모형 차수(p,d,q) 식별 근거 서술
- ④ 모형 적합 및 요약(AIC 등으로 모형 비교)
- ⑤ 잔차 진단(Ljung-Box)으로 모형 적합성 확인
- ⑥ 미래 예측 및 결과 해석

실기유형 연습문제 (10) — ARIMA 를 활용한 매출 예측

※ 아래 문제는 실제 시행된 시험의 출제 패턴(월별 매출액, 수요량 등 시계열 데이터를 활용한 예측 구조)을 반영하여 재구성한 기출유형 연습문제입니다.

[문제]

다음은 어느 프랜차이즈의 36개월간 월별 매출액 데이터(sales.csv, 컬럼: date, sales)이다.

(6-1) 시계열 데이터를 시각화하고, 정상성 여부를 ADF 검정으로 확인하시오. (6-2) 필요시 차분을 수행하고, ACF/PACF를 근거로 적절한 ARIMA 모형의 차수를 식별하시오. (6-3) ARIMA 모형을 적합시키고 잔차를 진단한 뒤, 향후 6개월의 매출액을 예측하시오.

모범 답안

(6-1) 시각화 및 정상성 검정

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller

df = pd.read_csv('sales.csv', parse_dates=['date'], index_col='date')

# 1) 원 시계열 시각화
plt.figure(figsize=(10,4))
plt.plot(df['sales'])
plt.title('월별 매출액 추이')
plt.xlabel('연월')
plt.ylabel('매출액')
plt.show()

# 2) ADF 정상성 검정
def adf_test(series, name=''):
    result = adfuller(series.dropna())
    print(f"[{name}] ADF 통계량: {result[0]:.4f}, p-value: {result[1]:.4f}")
    return result[1]

p_value = adf_test(df['sales'], '원 시계열')
```

[서술] 원 시계열을 시각화한 결과 시간이 지남에 따라 **완만한 상승 추세**가 관찰되며, 일부 구간에서 반복적인 패턴(계절성 가능성)도 함께 관찰된다. ADF 검정 결과 p-value가 0.05보다 크게 산출되어(비정상 상황을 가정), **귀무가설(단위근이 존재한다, 즉 비정상 시계열이다)**을 기각하지 못하므로 해당 시계열은 **비정상(non-stationary)**인 것으로 판단하였다.

(6-2) 차분 및 모형 차수 식별

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

# 1차 차분 수행
df['sales_diff'] = df['sales'].diff().dropna()

p_value_diff = adf_test(df['sales_diff'].dropna(), '1차 차분 후')

# ACF/PACF 플롯으로 p, q 차수 식별
fig, axes = plt.subplots(1, 2, figsize=(12,4))
plot_acf(df['sales_diff'].dropna(), ax=axes[0], lags=20)
```

```

plot_pacf(df['sales_diff'].dropna(), ax=axes[1], lags=20)
axes[0].set_title('ACF (차분 후)')
axes[1].set_title('PACF (차분 후)')
plt.show()

```

[서술] 1차 차분 수행 후 ADF 검정을 재실행한 결과 p-value가 0.05보다 작게 산출되어 **귀무가설을 기각, 즉 1차 차분($d=1$) 후에는 정상성을 확보**한 것으로 판단하였다. 차분된 시계열의 ACF 플롯을 확인한 결과 시차 1 이후 값이 빠르게 절단(cut-off)되는 패턴이, PACF는 시차 1~2에서 유의한 값을 보이다 완만하게 감소하는 패턴이 관찰되어, **MA(1) 성분과 소폭의 AR 성분이 혼재된 형태로** 판단하였다. 이를 근거로 **ARIMA(1,1,1)**을 우선 후보 모형으로 설정하되, AIC 기준으로 인접한 여러 모형과 비교하여 최종 차수를 확정하였다.

(6-3) 모형 적합, 잔차 진단, 예측

```

from statsmodels.tsa.arima.model import ARIMA
from statsmodels.stats.diagnostic import acorr_ljungbox

# 여러 후보 모형을 AIC 기준으로 비교
candidates = [(1,1,0), (0,1,1), (1,1,1), (2,1,1)]
aic_results = []
for order in candidates:
    model = ARIMA(df['sales'], order=order)
    fitted = model.fit()
    aic_results.append({'order': order, 'AIC': fitted.aic})

aic_df = pd.DataFrame(aic_results).sort_values('AIC')
print(aic_df)

# AIC가 가장 낮은 모형을 최종 모형으로 선정 (예: ARIMA(1,1,1)이 최적이라 가정)
best_model = ARIMA(df['sales'], order=(1,1,1))
best_fitted = best_model.fit()
print(best_fitted.summary())

# 잔차 진단: Ljung-Box 검정 (잔차의 자기상관 여부 확인)
lb_test = acorr_ljungbox(best_fitted.resid, lags=[10], return_df=True)
print(lb_test)

# 잔차 시각화
plt.figure(figsize=(10,4))
plt.plot(best_fitted.resid)
plt.title('잔차 플롯')
plt.axhline(0, color='red', linestyle='--')
plt.show()

# 향후 6개월 예측
forecast = best_fitted.get_forecast(steps=6)
forecast_mean = forecast.predicted_mean
conf_int = forecast.conf_int()

print(forecast_mean)

```

```
plt.figure(figsize=(10,4))
plt.plot(df['sales'], label='실제값')
plt.plot(forecast_mean, label='예측값', color='red')
plt.fill_between(conf_int.index, conf_int.iloc[:,0], conf_int.iloc[:,1], color='pink',
alpha=0.3)
plt.legend()
plt.title('ARIMA(1,1,1) 향후 6개월 매출 예측')
plt.show()
```

[서술] AIC 기준으로 여러 후보 모델을 비교한 결과 **ARIMA(1,1,1)의 AIC가 가장 낮게 나타나 최적 모형으로 선정**하였다. Ljung-Box 검정 결과 p-value가 0.05보다 크게 산출되어(예시 상황, 귀무가설(잔차에 자기상관이 없다, 즉 잔차가 백색잡음이다)을 기각하지 못하므로 잔차에 유의미한 패턴이 남아있지 않은 것으로 판단하였다. 이는 **모형이 시계열의 패턴을 적절히 설명하고 있음을 의미**한다. (만약 Ljung-Box 검정에서 p-value가 0.05보다 작게 나왔다면 "잔차에 자기상관이 남아있어 모형의 차수를 재조정하거나 계절성(SARIMA) 요소를 추가로 고려해야 한다"고 서술해야 한다.)

향후 6개월 예측 결과, 매출액은 기존의 상승 추세를 유지하며 완만하게 증가할 것으로 예측되었다. 신뢰구간이 예측 시점이 멀어질수록 넓어지는 것은 **시계열 예측의 일반적인 특성으로, 먼 미래일수록 불확실성이 커짐**을 반영한 결과로 해석된다.

참고: 계절성이 있는 경우 — SARIMA 적용 근거 서술 예시

[서술 예시] 만약 원 시계열에서 **12개월 주기의 반복적인 패턴(계절성)**이 뚜렷하게 관찰된다면, 단순 ARIMA보다 계절 성분을 반영할 수 있는 **SARIMA(p,d,q)(P,D,Q)m** 모형(m=12)을 고려해야 한다. 이 경우 계절차분(seasonal differencing)을 추가로 수행하여 계절성을 제거한 뒤, 계절 ACF/PACF 패턴을 통해 계절 차수(P,D,Q)를 추가로 식별하는 절차가 필요하다.

```
from statsmodels.tsa.statespace.sarimax import SARIMAX

sarima_model = SARIMAX(df['sales'], order=(1,1,1), seasonal_order=(1,1,1,12))
sarima_fitted = sarima_model.fit()
print(sarima_fitted.summary())
```

이 파트에서 감점을 피하는 핵심 포인트

- **✗** 정상성 검정 없이 바로 ARIMA 적합 → 반드시 ADF 검정으로 정상성을 먼저 확인하고 그 결과(p-value)를 근거로 차분 여부를 결정
- **✗** ACF/PACF 없이 임의로 (p,d,q)를 설정 → 그래프의 패턴(절단 vs 점진적 감소)을 근거로 차수를 설명하거나, 최소한 여러 후보를 AIC 로 비교했다는 절차를 보여줄 것
- **✗** 모형 적합 후 잔차 진단 생략 → Ljung-Box 검정으로 잔차의 백색잡음 여부를 확인하는 것이 모형의 타당성을 입증하는 핵심 근거
- **✗** 예측값만 제시하고 해석 없음 → 예측 결과가 보여주는 추세와 신뢰구간이 넓어지는 이유까지 서술

- ☒ 계절성이 있어 보이는 데이터라면 SARIMA 를 언급만 해도(직접 구현하지 못하더라도) 개념적 이해도를 보여줄 수 있어 가점

[사용자] 네

ADP 실기시험 대비 자료 (7) — 텍스트마이닝(비정형 데이터 처리)

텍스트마이닝은 매회 고정 출제되지는 않지만, 출제될 경우 한국어 텍스트 처리 특유의 절차(형태소 분석)를 다룰 줄 아는지가 관건입니다. 정형 데이터 문제보다 코드량은 적어도, 전처리 근거와 결과 해석을 놓치면 감점되는 구조는 동일합니다.

핵심 체크리스트

항목	확인할 것
텍스트 정제	특수문자/숫자/이모티콘 제거 (정규표현식 re 활용)
형태소 분석	한국어는 konlpy(Okt, Mecab 등)로 명사/형태소 추출 — 영어와 달리 띄어쓰기만으로 토큰화 불가능을 서술
불용어 제거	조사, 대명사 등 분석에 의미 없는 단어 제거 (불용어 사전 활용 또는 직접 정의)
벡터화	CountVectorizer(BoW) vs TfidfVectorizer(TF-IDF) — 목적에 따라 선택 근거 서술
빈도 분석/시각화	단어 빈도표, 워드클라우드(WordCloud) — 한글 폰트 설정 필수 언급
감성분석	감성사전 기반 vs 기계학습 기반 — 데이터에 레이블(긍/부정) 존재 여부에 따라 방법 선택
분류 모델과 결합	TF-IDF 벡터를 지도학습 모델(나이브베이즈, 로지스틱회귀 등)의 입력으로 사용하는 파이프라인 구성

답안 작성 표준 틀

- ① 텍스트 정제(전처리) 및 근거 서술
- ② 형태소 분석/토큰화 수행
- ③ 불용어 제거
- ④ 벡터화(BoW 또는 TF-IDF) 및 선택 근거
- ⑤ 분석(빈도/시각화/감성분석/분류모델) 수행
- ⑥ 결과 해석 및 실무적 시사점 도출

실기유형 연습문제 (11) — 리뷰 텍스트 전처리 및 TF-IDF 기반 핵심어 추출

※ 아래 문제는 실제 시행된 시험의 출제 패턴(상품 리뷰, SNS 텍스트 데이터를 활용한 전처리·핵심어 추출·감성분류 구조)을 반영하여 재구성한 기출유형 연습문제입니다.

[문제]

다음은 어느 온라인 쇼핑몰의 상품 리뷰 데이터(review.csv, 컬럼: review_text, rating(1~5점))이다.

(7-1) 리뷰 텍스트를 전처리(정제, 형태소 분석, 불용어 제거)하시오. **(7-2)** TF-IDF를 활용하여 리뷰 전체에서 중요도가 높은 핵심 단어 상위 10개를 추출하고 워드클라우드를 시각화하시오. **(7-3)** rating이 4점 이상이면 긍정(1), 2점 이하면 부정(0)으로 정의하고, TF-IDF 벡터를 활용한 감성 분류 모델을 구축하시오.

모범 답안

(7-1) 텍스트 전처리

```
import pandas as pd
import re
from konlpy.tag import Okt

df = pd.read_csv('review.csv')

# 1) 텍스트 정제: 한글, 공백만 남기고 특수문자/숫자/영어 제거
def clean_text(text):
    text = re.sub(r'^가-힣\s', '', str(text))
    text = re.sub(r'\s+', ' ', text).strip()
    return text

df['cleaned'] = df['review_text'].apply(clean_text)

# 2) 형태소 분석 (명사 추출) - 한국어는 교착어 특성상 띄어쓰기만으로는
# 의미 단위 토큰화가 불가능하므로 형태소 분석기가 필수적으로 요구됨
okt = Okt()
df['nouns'] = df['cleaned'].apply(lambda x: okt.nouns(x))

# 3) 불용어 제거
stopwords = ['것', '수', '등', '들', '이', '저', '제품', '진짜', '너무']
df['tokens'] = df['nouns'].apply(lambda x: [w for w in x if w not in stopwords and len(w) > 1])

print(df[['review_text', 'tokens']].head())
```

[서술] 한국어는 영어와 달리 조사·어미가 결합되는 교착어이므로, 단순 띄어쓰기 기준 토큰화만으로는 의미 있는 단어 단위를 추출할 수 없다. 이에 konlpy의 **Okt 형태소 분석기**를 사용하여 **명사(nouns)**를 추출하였다. 이후 분석에 의미가 없는 조사성 표현이나 지시대명사, 그리고 리뷰 데이터에 지나치게 빈번하게 등장하여 변별력이 낮은 단어("제품", "진짜" 등)를 **불용어로 정의하여 제거**하였다. 또한 한 글자 단어는 의미 파악이 어려운 경우가 많아 $\text{len}(w) > 1$ 조건으로 함께 필터링하였다.

(7-2) TF-IDF 핵심어 추출 및 워드클라우드

```
from sklearn.feature_extraction.text import TfidfVectorizer
from wordcloud import WordCloud
import matplotlib.pyplot as plt

# 토큰 리스트를 다시 문자열로 결합 (Vectorizer 입력 형태에 맞춤)
df['tokens_str'] = df['tokens'].apply(lambda x: ' '.join(x))

# TF-IDF 벡터화
# CountVectorizer(단순 빈도)가 아닌 TfidfVectorizer를 사용한 이유:
# 모든 리뷰에 공통적으로 자주 등장하는 단어의 가중치는 낮추고,
# 특정 리뷰(또는 일부 리뷰군)를 구별짓는 단어에 더 높은 가중치를 부여하기 위함
tfidf = TfidfVectorizer(max_features=1000)
tfidf_matrix = tfidf.fit_transform(df['tokens_str'])

# 전체 문서에서 TF-IDF 점수 합이 높은 상위 10개 단어 추출
tfidf_sum = tfidf_matrix.sum(axis=0)
words = tfidf.get_feature_names_out()
word_scores = pd.Series(tfidf_sum.A1, index=words).sort_values(ascending=False)
print(word_scores.head(10))

# 워드클라우드 시각화 (한글 폰트 경로 지정 필수)
wc = WordCloud(
    font_path='C:/Windows/Fonts/malgun.ttf', # 실기시험 환경의 한글 폰트 경로로 대체 필요
    width=800, height=400, background_color='white'
).generate_from_frequencies(word_scores.to_dict())

plt.figure(figsize=(10,5))
plt.imshow(wc, interpolation='bilinear')
plt.axis('off')
plt.title('리뷰 핵심 단어 워드클라우드')
plt.show()
```

[서술] 단순 등장 빈도만 반영하는 CountVectorizer(BoW) 대신 **TF-IDF**를 사용한 이유는, 모든 리뷰에 공통으로 자주 등장하는 단어(예: 배송 관련 상투적 표현)의 가중치를 낮추고, **특정 상품·특성을 실제로 대표하는 단어에 더 높은 가중치를 부여하기 위함**이다. TF-IDF 점수 상위 단어들을 확인한 결과 "배송", "품질", "디자인" 등이 상위에 위치하는 것으로 나타났다(예시). 이는 **고객들이 리뷰에서 가장 자주, 그리고 변별력 있게 언급하는 핵심 관심사가 배송 속도와 상품 품질임을 시사**하며, 워드클라우드 시각화를 통해서도 이러한 핵심 키워드가 시각적으로 두드러지게 표현되었다. **한글이 깨지지 않도록 워드클라우드 생성 시 한글 폰트 경로를 반드시 지정해야** 한다는 점도 실무적으로 중요한 포인트이다.

(7-3) TF-IDF 기반 감성 분류 모델

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
```

```

# 레이블 생성: rating 4점 이상=긍정(1), 2점 이하=부정(0), 3점은 중립으로 분석 대상에서 제외
df_binary = df[df['rating'] != 3].copy()
df_binary['label'] = (df_binary['rating'] >= 4).astype(int)

X = tfidf.transform(df_binary['tokens_str']) # 앞서 학습된 tfidf로 transform
y = df_binary['label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# 나이브베이지: 텍스트(단어 빈도 기반) 분류에서 전통적으로 널리 쓰이는 베이스라인 모델
nb = MultinomialNB()
nb.fit(X_train, y_train)
pred_nb = nb.predict(X_test)

# 로지스틱회귀: TF-IDF 벡터에도 적용 가능하며 계수 해석이 용이
lr = LogisticRegression(max_iter=1000)
lr.fit(X_train, y_train)
pred_lr = lr.predict(X_test)

print("[나이브베이지]")
print(classification_report(y_test, pred_nb))
print("[로지스틱회귀]")
print(classification_report(y_test, pred_lr))

```

[서술] rating이 3점인 리뷰는 긍정도 부정도 아닌 **중립적 성격이 강해 이진분류의 경계를 모호하게 만들 수 있으므로 분석 대상에서 제외**하였다. 감성분류 모델로는 텍스트(단어 빈도/TF-IDF) 분류에서 전통적으로 우수한 성능을 보이는 **나이브베이지**와, 계수를 통한 해석이 용이한 **로지스틱회귀**를 함께 비교하였다. 두 모델의 성능을 비교한 결과(예시), 나이브베이지가 근소하게 더 높은 정확도를 보였으며, 이는 **TF-IDF와 같은 희소(sparse)하고 고차원인 텍스트 벡터에 나이브베이지의 조건부 독립 가정이 비교적 잘 부합하기** 때문으로 해석할 수 있다.

[한계 서술] 다만 본 모델은 단어의 등장 순서나 문맥을 전혀 고려하지 않는 **Bag-of-Words** 기반 접근이라는 한계가 있다. 예를 들어 "배송이 빠르지 않다"와 "배송이 빠르다"를 문맥상 구분하지 못할 수 있으며, 이를 보완하기 위해서는 **N-gram 적용, 또는 문맥을 반영하는 임베딩 기반 딥러닝 모델(Word2Vec, BERT 등)**을 추가로 고려할 수 있다는 점을 개선방안으로 제시한다.

이 파트에서 감점을 피하는 핵심 포인트

- **✗** 한국어 텍스트에 영어처럼 단순 split()으로 토큰화 → 반드시 형태소 분석기(konlpy 등) 사용 근거를 서술
- **✗** 불용어 제거 없이 바로 벡터화 → 조사, 의미 없는 고빈도 단어가 결과를 왜곡할 수 있음을 언급

- **✗** CountVectorizer와 TF-IDF의 차이를 설명하지 않고 그냥 사용 → 왜 TF-IDF를 선택했는지(공통 단어 가중치 감소) 반드시 서술
- **✗** 워드클라우드에서 한글 폰트 미지정으로 깨짐 → 실기시험에서 실제로 자주 발생하는 실수이므로 폰트 경로 지정을 답안에 명시
- **✗** 감성분류 모델의 한계(문맥 미반영)를 언급하지 않음 → BoW/TF-IDF 기반 접근의 근본적 한계를 아는 것이 심화 이해도를 보여주는 포인트
- **✓** 나이브베이즈 외에 로지스틱회귀 등 추가 모델과 비교하면 분류 파트와 마찬가지로 가점

[사용자] 네

ADP 실기시험 대비 자료 (8) — 종합 모의고사

지금까지 배운 모든 기법(전처리, 통계검정, 분류/회귀 모델링, 군집분석, 시계열, 텍스트마이닝)을 실제 시험과 동일한 형태로 통합한 모의고사입니다. 실전처럼 대문제 3개로 구성하였으며, 각 대문제는 서로 다른 데이터셋과 여러 소문제로 이루어져 있습니다.

🕒 **실전 팁:** 실제 시험은 240분(4시간)입니다. 대문제당 약 70~75분을 배분하고, 남은 시간은 검토와 서술 보완에 활용하는 것을 권장합니다. 시간이 부족할 경우 **코드보다 서술(가설, 근거, 해석)을 먼저 채우는 것이 부분점수 확보에 유리합니다.**

대문제 1 — 병원 환자 데이터 (전처리 + 통계검정 + 분류모델링)

※ 아래 문제는 실제 시행된 시험의 출제 패턴(의료/헬스케어 데이터를 활용한 전처리→검정→분류 통합형 구조)을 반영하여 재구성한 기출유형 연습문제입니다.

[데이터 설명] patient.csv — 당뇨병 재발 위험을 예측하기 위한 환자 데이터

- age(나이), bmi(체질량지수, 일부 결측), blood_pressure(혈압), treatment_group(치료법 A/B/C), glucose_level(혈당), readmit(재입원 여부, 1=재입원/0=미재입원)

(1-1) 결측치와 이상치를 처리하시오. **(1-2)** 치료법(A/B/C)에 따라 혈당 수치(glucose_level)에 유의한 차이가 있는지 유의수준 5%에서 검정하시오. **(1-3)** 재입원 여부(readmit)를 예측하는 분류모델을 구축하고, 가장 중요한 예측 변수를 제시하시오.

모범 답안

(1-1) 전처리

```
import pandas as pd
import numpy as np
from scipy import stats

df = pd.read_csv('patient.csv')
```

```

# 결측치 확인
print(df.isnull().sum())

# bmi는 연속형이며 나이/혈압 등과 관련성이 있을 수 있으므로 단순 평균 대체보다
# treatment_group별 중앙값으로 대체하여 집단 특성을 보존
df['bmi'] = df.groupby('treatment_group')['bmi'].transform(lambda x:
x.fillna(x.median()))

# 이상치(IQR) 확인 및 캡핑
Q1, Q3 = df['blood_pressure'].quantile([0.25, 0.75])
IQR = Q3 - Q1
lower, upper = Q1 - 1.5*IQR, Q3 + 1.5*IQR
df['blood_pressure'] = df['blood_pressure'].clip(lower, upper)

```

[서술] bmi의 결측치는 전체 평균이 아닌 **treatment_group별 중앙값으로 대체**하여, 치료군 간 특성 차이가 있을 수 있다는 점을 반영하였다. blood_pressure의 이상치는 IQR 기준으로 확인 후 **상하한 경계값으로 캡핑**하여 극단값의 영향을 완화하면서도 표본 손실을 방지하였다.

(1-2) 통계검정 (일원배치 ANOVA)

```

groups = [df[df['treatment_group']==g]['glucose_level'] for g in ['A','B','C']]

# 정규성/등분산성 확인
for g, data in zip(['A','B','C'], groups):
    print(g, stats.shapiro(data))
print("Levene:", stats.levene(*groups))

f_stat, p_value = stats.f_oneway(*groups)
print(f"F={f_stat:.4f}, p={p_value:.6f}")

from statsmodels.stats.multicomp import pairwise_tukeyhsd
tukey = pairwise_tukeyhsd(df['glucose_level'], df['treatment_group'], alpha=0.05)
print(tukey)

```

[서술] H0: 세 치료법 간 평균 혈당 수치는 같다 / H1: 적어도 한 그룹은 다르다. 유의수준 5%에서 ANOVA 결과 $p\text{-value} < 0.05$ 로 **귀무가설을 기각**, 치료법에 따라 혈당 수치에 유의한 차이가 존재함을 확인하였다. Tukey 사후검정 결과 **치료법 B가 A, C와 유의한 차이를 보여**, 혈당 조절 측면에서 B가 상대적으로 효과적임을 시사한다.

(1-3) 분류모델링

```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score, roc_auc_score, classification_report

df_enc = pd.get_dummies(df, columns=['treatment_group'], drop_first=True)
X = df_enc.drop('readmit', axis=1)

```

```

y = df_enc['readmit']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, stratify=y,
random_state=42)
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

rf = RandomForestClassifier(n_estimators=200, random_state=42)
rf.fit(X_train_s, y_train)
pred = rf.predict(X_test_s)
proba = rf.predict_proba(X_test_s)[:,:1]
print("F1:", f1_score(y_test, pred), "AUC:", roc_auc_score(y_test, proba))

importance = pd.Series(rf.feature_importances_,
index=X.columns).sort_values(ascending=False)
print(importance)

```

[서술] 재입원 여부는 임상적으로 불균형 데이터일 가능성이 높아 stratify=y로 층화추출하고, **F1-score와 AUC 중심으로 평가**하였다. 변수 중요도 확인 결과 **glucose_level(혈당)과 bmi**가 재입원 예측에 가장 중요한 변수로 나타나, 혈당 관리와 체중 관리가 재입원 예방의 핵심 관리 포인트임을 시사한다.

대문제 2 — 매장 매출 데이터 (회귀 + 군집분석)

[데이터 설명] store.csv — 프랜차이즈 매장별 데이터

- store_size(매장면적), staff_count(직원 수), foot_traffic(유동인구지수), marketing_cost(마케팅비), monthly_revenue(월매출, 목표변수), avg_visit(방문당 평균 체류시간)

(2-1) 다중공선성을 점검하고 월매출을 예측하는 회귀모델을 구축·비교하시오. **(2-2)** 매장을 특성에 따라 군집화하고, 군집별 특성과 시사점을 제시하시오.

모범 답안

(2-1) 회귀 모델링

```

from statsmodels.stats.outliers_influence import variance_inflation_factor
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score

X = df.drop('monthly_revenue', axis=1)
y = df['monthly_revenue']

vif = pd.DataFrame({'변수': X.columns,
                    'VIF': [variance_inflation_factor(X.values, i) for i in
range(X.shape[1])]}])
print(vif)

```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
scaler = StandardScaler()
X_train_s, X_test_s = scaler.fit_transform(X_train), scaler.transform(X_test)

for name, model in [('선형회귀', LinearRegression()), ('Ridge', Ridge(alpha=1.0)),
                    ('랜덤포레스트', RandomForestRegressor(random_state=42))]:
    model.fit(X_train_s, y_train)
    pred = model.predict(X_test_s)
    print(name, "RMSE:", np.sqrt(mean_squared_error(y_test, pred)), "R2:",
          r2_score(y_test, pred))

```

[서술] VIF 확인 결과 store_size와 foot_traffic이 다소 높게 나올 경우(예시), 규제모델인 **Ridge**를 함께 비교 대상에 포함하는 것이 타당하다. 성능 비교 결과 랜덤포레스트가 R²와 RMSE 모두 우수하였으나, **선형회귀/Ridge 계수를 통해 foot_traffic(유동인구)이 매출에 가장 큰 양(+)**의 영향을 미친다는 해석 가능한 결과도 함께 제시하여 실무적 설명력을 보완하였다.

(2-2) 군집분석

```

from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

cluster_vars = ['store_size', 'foot_traffic', 'marketing_cost', 'avg_visit']
X_c = scaler.fit_transform(df[cluster_vars])

for k in range(2,6):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X_c)
    print(k, silhouette_score(X_c, labels))

km = KMeans(n_clusters=3, random_state=42, n_init=10)
df['cluster'] = km.fit_predict(X_c)
print(df.groupby('cluster')[cluster_vars + ['monthly_revenue']].mean())

```

[서술] 실루엣 계수가 K=3에서 가장 높게 나타나 3개 군집으로 최종 결정하였다. 프로파일링 결과 ① 대형·고유동인구·고매출 매장(핵심 상권형), ② 중소형·저마케팅비·중간매출 매장(안정형), ③ 저유동인구·저매출 매장(개선 필요형)으로 유형화할 수 있으며, ③유형 매장에는 마케팅비 재배분이 나 입지 재검토가 필요하다는 실무적 제언을 제시할 수 있다.

대문제 3 — 콜센터 상담 데이터 (시계열 + 텍스트마이닝)

[데이터 설명] callcenter.csv — 일별 상담 건수(date, call_count) 및 상담 내용 텍스트(complaint_text)

(3-1) 일별 상담 건수의 정상성을 검정하고 향후 7일을 예측하시오. (3-2) 상담 내용 텍스트를 전처리하여 핵심 불만 키워드를 추출하시오.

모범 답안

(3-1) 시계열 예측

```
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA

ts = pd.read_csv('callcenter.csv', parse_dates=['date'], index_col='date')['call_count']

result = adfuller(ts)
print("ADF p-value:", result[1])

# 비정상일 경우 차분
ts_diff = ts.diff().dropna()
print("차분 후 ADF p-value:", adfuller(ts_diff)[1])

model = ARIMA(ts, order=(1,1,1)).fit()
forecast = model.get_forecast(steps=7)
print(forecast.predicted_mean)
```

[서술] ADF 검정 결과 원 시계열은 비정상으로 판단되어 1차 차분 후 정상성을 확보하였다. ARIMA(1,1,1) 적합 후 향후 7일 예측을 수행하였으며, 예측 상담 건수가 증가 추세를 보인다면 **상담 인력 사전 증원 계획 수립**이 필요하다는 실무적 시사점을 제시할 수 있다.

(3-2) 텍스트마이닝

```
from konlpy.tag import Okt
from sklearn.feature_extraction.text import TfidfVectorizer

okt = Okt()
df_text = pd.read_csv('callcenter.csv')
df_text['nouns'] = df_text['complaint_text'].apply(lambda x: ' '.join(okt.nouns(str(x))))

tfidf = TfidfVectorizer(max_features=500)
tfidf_matrix = tfidf.fit_transform(df_text['nouns'])
scores = pd.Series(tfidf_matrix.sum(axis=0).A1, index=tfidf.get_feature_names_out())
print(scores.sort_values(ascending=False).head(10))
```

[서술] 형태소 분석을 통한 명사 추출 후 TF-IDF로 핵심 키워드를 확인한 결과 "지연", "환불", "불친절" 등이 상위에 위치한다면(예시), 이는 **배송 지연 및 응대 품질이 주요 불만 요인**임을 시사하며, 시계열 예측에서 확인된 상담량 급증 시점과 연결지어 **원인-현상을 함께 설명하는 종합적 결론**을 제시할 수 있다는 점에서 대문제 간 연계 해석이 가능하다.

모의고사를 통해 확인해야 할 최종 점검 사항

- 모든 소문제에 가설/근거/해석 문장을 빠짐없이 작성했는가
- 전처리 단계에서 선택의 이유를 서술했는가
- 통계검정에서 H0/H1/유의수준/결론을 형식에 맞게 작성했는가
- 모델링에서 2 개 이상 모델 비교 + 평가지표 선택 근거를 제시했는가

- Data Leakage(스케일러 fit 시점) 실수가 없는가
- 그래프에 제목/축 레이블이 있는가
- 마지막에 실무적 시사점/한계/개선방안을 한 줄이라도 남겼는가

이것으로 ADP 실기시험 대비 8단계(개요/전략, 전처리&EDA, 통계검정, 분류, 회귀, 군집분석, 시계열, 텍스트마이닝, 종합모의고사) 전체 커리큘럼을 마쳤습니다.